LAPORAN TUGAS BESAR

Milestone 1: Sistem Terdistribusi Dasar

IF4031 Arsitektur Aplikasi Terdistribusi

Semester I Tahun Akademik 2026/2027

Disusun oleh

Rafif Farras 13523095

Azfa Radhiyya Hakim 13523115

Samantha Laqueenna Ginting 13523138

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Pernyataan Integritas Akademik dan Penggunaan AI

Rafif Farras (13523095)	Azfa Radhiyya Hakim (13523115)	Samantha Laqueenna Ginting (13523138)

Pembagian Tugas

Anggota	Kontribusi
Rafif Farras (13523095)	<Kontribusi Anggota 1>
Azfa Radhiyya Hakim (13523115)	<Kontribusi Anggota 2>
Samantha Laqueenna Ginting (13523138)	<Kontribusi Anggota 3>

Daftar Isi

Pernyataan Integritas Akademik dan Penggunaan AI

Pembagian Tugas

1 Pendahuluan **1**

1.1 Latar Belakang . . . 1

1.2 Tujuan . . . 1

1.3 Ruang Lingkup Milestone 1 . . . 1

1.4 Batasan Sistem . . . 2

1.5 Struktur Laporan . . . 2

2 Arsitektur Sistem **2**

2.1 Gambaran Umum . . . 2

2.2 Komponen dan Batas Service . . . 4

2.3 Alur Komunikasi dan Data . . . 4

2.4 Teknologi dan Konfigurasi . . . 4

3 Problem 1: Arsitektur dan Interoperabilitas Data **5**

3.1 Rancangan Solusi . . . 5

3.1.1 Aturan Pemetaan ke HazardEvent . . . 5

3.1.2 Kebijakan Evolusi Skema . . . 6

3.2 Alternatif dan Trade-off . . . 6

3.3 Proof of Concept (PoC) . . . 7

4 Problem 2: Concurrency, Availability, dan Graceful Degradation **9**

4.1 Rancangan Solusi . . . 9

4.2 Alternatif dan Trade-off . . . 10

4.3 Batasan dan Bukti Penyelesaian . . . 10

5 Problem 3: Service-to-Service Authentication dan Trust **11**

5.1 Rancangan Solusi . . . 11

5.2 Alternatif dan Trade-off . . . 12

5.3 Batasan yang Disadari . . . 12

5.4 Bukti Penyelesaian 13
5.5 Jawaban atas Butir “Di Laporan” 14

6 Problem 4: Arsitektur Microservice dan Flexible Storage 14
6.1 Rancangan Solusi 14
6.2 Alternatif dan Trade-off 15
6.3 Batasan yang Disadari 15
6.4 Bukti Penyelesaian 15

7 Problem 5: Fan-out Event Distribution dan Pub-sub 16
7.1 Rancangan Solusi 16
7.2 Alternatif dan Trade-off 17
7.3 Batasan yang Disadari 17
7.4 Bukti Penyelesaian 18
7.5 Jawaban atas Butir “Di Laporan” 19

A Bukti Pengujian Tambahan 19

1 Pendahuluan

1.1 Latar Belakang

Penanggulangan bencana melibatkan lembaga-lembaga yang bekerja secara otonom, memiliki sumber data, format informasi, dan karakteristik infrastruktur yang berbeda. Dalam skenario tugas ini, BMKG menyediakan informasi kejadian seismik dan potensi tsunami, sedangkan PVMBG menyediakan laporan aktivitas vulkanik. BNPB membutuhkan kedua aliran informasi tersebut untuk membangun gambaran situasi yang terpadu dan menyajikannya kepada pihak yang berwenang. Namun, perbedaan format data, latensi layanan, dan batas kepercayaan antarlembaga membuat integrasi langsung menjadi tidak memadai.

Masalah tersebut tidak hanya berkaitan dengan pertukaran data. Ketika satu layanan sumber melambat atau gagal, layanan lain tetap dapat menghasilkan data yang bernilai bagi pengguna. Demikian pula, lonjakan permintaan pada saat status bencana meningkat tidak boleh membuat seluruh sistem berhenti merespons. Sistem juga perlu membedakan hak akses antara Media, Tim Lapangan, dan BNPB Internal agar data mentah seperti koordinat presisi serta atribut spesifik sumber tidak bocor kepada pihak yang hanya membutuhkan ringkasan.

Oleh sebab itu, proyek ini membangun proof-of-concept platform koordinasi kebencanaan BNPB sebagai aplikasi terdistribusi. Platform menyatukan data heterogen dari BMKG dan PVMBG melalui komunikasi jaringan, mengubahnya ke format kanonik *HazardEvent*, menyimpannya pada *Canonical Store*, dan mendistribusikan informasi tersebut melalui API serta mekanisme *publish-subscribe*.

1.2 Tujuan

Tujuan utama pengembangan sistem adalah membuktikan bahwa beberapa layanan otonom dapat diintegrasikan menjadi satu platform koordinasi tanpa menghilangkan batas kepemilikan dan karakteristik masing-masing layanan. Secara khusus, Milestone 1 bertujuan untuk:

1. menyatukan data seismik dan vulkanik dari dua instansi ke dalam format kanonik yang konsisten serta tetap menerima perubahan skema aditif;
2. menjaga ketersediaan data penting melalui pemrosesan asinkron, pembatasan konkurensi, dan degradasi layanan yang eksplisit ketika salah satu sumber tidak tersedia;
3. menerapkan pemisahan domain kepercayaan untuk komunikasi antarlayanan dan akses client downstream;
4. memisahkan komponen ke dalam layanan dan container yang dapat dijalankan secara independen, dengan penyimpanan yang mendukung atribut dinamis; serta
5. mendistribusikan event baru ke beberapa consumer melalui message broker tanpa membuat Aggregator bergantung langsung pada consumer tersebut.

1.3 Ruang Lingkup Milestone 1

Ruang lingkup implementasi dibatasi pada dua instansi sumber dan dua jenis bencana. BMKG disimulasikan sebagai sumber *SeismicEvent* dan *TsunamiWarning*; PVMBG disimulasikan

sebagai sumber *VolcanicReport*. Kedua sumber bersifat pasif dan hanya merespons polling berkala dari Aggregator BNPB. Data dari sumber tidak dibagikan melalui pemanggilan fungsi langsung atau library *business logic* bersama, melainkan melalui API jaringan sesuai kontrak layanan.

Komponen BNPB dalam Milestone 1 meliputi Aggregator atau gateway, Canonical Store, Client-Facing API, Auth/Token Service, message broker, dan sedikitnya dua event consumer. Aggregator melakukan polling, pemetaan, penyimpanan, serta publikasi event baru. Canonical Store hanya dimiliki dan diakses langsung oleh Aggregator; layanan lain memperoleh data melalui API Aggregator. Client-Facing API bersifat *read-only* dan menerapkan hak akses berdasarkan scope token.

Hak akses client dibagi menjadi tiga tingkat. Media hanya menerima field berklasifikasi Ringkasan. Tim Lapangan memperoleh data lengkap untuk kebutuhan operasional, sedangkan BNPB Internal memiliki hak akses tertinggi. Access token bersifat berumur pendek dan harus dapat diperbarui melalui alur *refresh* tanpa login ulang. Batasan ini dibuat agar keamanan, ketahanan, dan distribusi data dapat diuji secara nyata pada arsitektur yang sama.

1.4 Batasan Sistem

Milestone 1 belum mencakup seluruh kebutuhan sistem kebencanaan nasional. Sistem tidak menangani jalur pelaporan dari lapangan ke BNPB, perubahan data oleh client downstream, maupun sumber bencana lain seperti kebakaran hutan dan hidrometeorologi. Fokusnya adalah fondasi interoperabilitas, availability, keamanan, fleksibilitas storage, dan distribusi event pada skala dasar.

Pengembangan untuk Milestone 2 berada di luar cakupan laporan ini. Tahap berikutnya akan menguji fondasi yang sama dalam kondisi yang lebih skalabel, termasuk banyak instance layanan, volume data yang meningkat drastis, dan kegagalan beberapa komponen secara bersamaan.

1.5 Struktur Laporan

Bab berikutnya menjelaskan arsitektur sistem, batas tiap service, serta teknologi yang digunakan. Setelah itu, setiap problem resmi Milestone 1 dibahas secara terpisah, mencakup rancangan solusi, alternatif dan trade-off, batasan yang disadari, serta bukti penyelesaian. Lampiran digunakan untuk menyimpan bukti pengujian tambahan seperti tangkapan layar, log ber-correlation ID, dan hasil pengukuran.

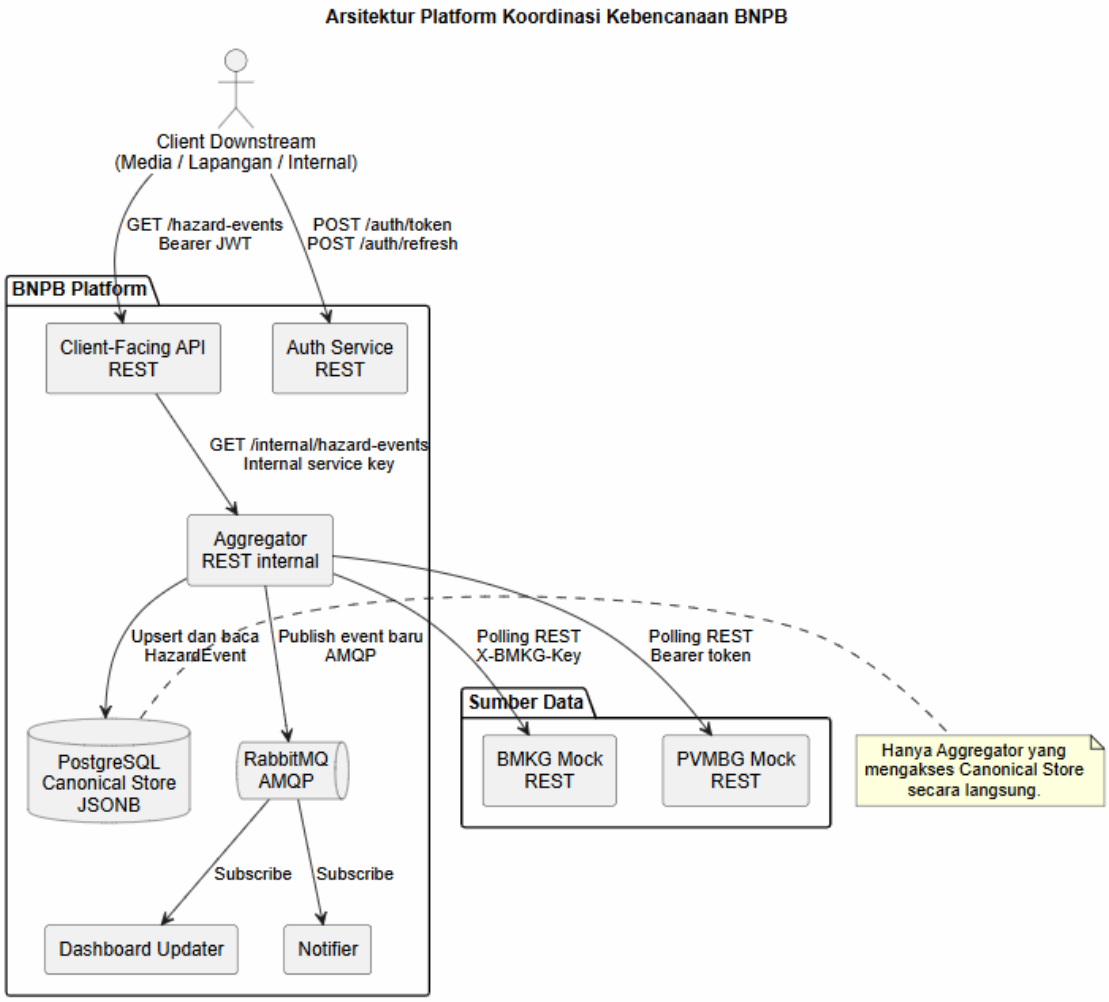
2 Arsitektur Sistem

2.1 Gambaran Umum

Sistem dibangun sebagai sekumpulan service yang dikemas dan dijalankan terpisah melalui Docker Compose. BMKG dan PVMBG berperan sebagai *mock service*. Aggregator melakukan

polling ke kedua sumber, memetakan respons ke format kanonik *HazardEvent*, menyimpannya pada Canonical Store, dan menerbitkan event baru ke message broker.

Client downstream tidak membaca database secara langsung. Permintaan data masuk melalui Client-Facing API, yang memverifikasi token client, menerapkan filter hak akses, lalu mengambil data dari API internal milik Aggregator. Event baru diterima consumer independen untuk kebutuhan dashboard dan notifikasi. Diagram arsitektur sistem ditunjukkan pada Gambar 1.



Gambar 1: Arsitektur platform koordinasi kebencanaan BNPB.

2.2 Komponen dan Batas Service

Tabel 1: Komponen utama sistem.

Komponen	**Antarmuka**	**Tanggung Jawab**
BMKG Mock	REST	Menyediakan data *SeismicEvent* dan *TsunamiWarning*; akses menggunakan `X-BMKG-Key`.
PVMBG Mock	REST	Menyediakan *VolcanicReport*, perubahan skema, dan simulasi outage; akses menggunakan *Bearer token*.
Aggregator	REST internal	Polling, mapping, penyimpanan Canonical Store, serta publikasi event baru.
PostgreSQL	Koneksi database internal	Menyimpan *HazardEvent* dengan atribut dinamis pada kolom `JSONB`; hanya Aggregator yang mengaksesnya langsung.
Client-Facing API	REST	Menyajikan data untuk client, memvalidasi JWT, dan menerapkan RBAC.
Auth Service	REST	Menerbitkan dan memperbarui token client.
RabbitMQ	AMQP	Broker distribusi event dengan pola fan-out.
Event Consumer	AMQP consumer	Menerima event baru secara independen untuk dashboard dan notifikasi.

2.3 Alur Komunikasi dan Data

Aggregator melakukan polling berkala melalui REST. Kredensial untuk BMKG berupa API key, sedangkan PVMBG menggunakan *Bearer token*. Respons sumber dipetakan ke *HazardEvent* dan disimpan oleh Aggregator ke PostgreSQL. Field dinamis disimpan pada `attributes` bertipe `JSONB`.

Setiap event baru yang tersimpan diterbitkan Aggregator ke RabbitMQ. Dashboard Updater dan Notifier berlangganan melalui broker; Aggregator tidak memanggil consumer secara langsung. Untuk alur baca, client memperoleh access token dari Auth Service dan mengirimkannya ke Client-Facing API. Setelah memvalidasi token dan scope, Client-Facing API memanggil endpoint internal Aggregator dengan *internal service key*. Dengan demikian, PostgreSQL bukan dependency langsung bagi Client-Facing API maupun consumer.

2.4 Teknologi dan Konfigurasi

Service aplikasi dibangun menggunakan Node.js, TypeScript, dan Fastify. Komunikasi sinkron menggunakan REST, sedangkan distribusi event memakai RabbitMQ melalui AMQP. Canonical Store menggunakan PostgreSQL dengan `JSONB` untuk menyimpan atribut sumber yang dinamis tanpa penambahan kolom baru.

Seluruh service dijalankan melalui `docker-compose.yml`. Port, URL service, credential, interval polling, parameter circuit breaker, koneksi database, koneksi broker, dan kunci

internal dikonfigurasi melalui environment variable.

3 Problem 1: Arsitektur dan Interoperabilitas Data

Pertanyaan Utama

Bagaimana Aggregator BNPB dapat menyatukan data BMKG dan PVMBG ke format kanonik yang tetap valid ketika PVMBG menambahkan field baru tanpa restart atau redeploy?

3.1 Rancangan Solusi

Sistem menggunakan pola integrasi *pull* melalui REST. Aggregator melakukan polling ke BMKG untuk data *SeismicEvent* dan *TsunamiWarning*, serta ke PVMBG untuk data *VolcanicReport*. Setiap respons dipetakan menjadi kontrak internal tunggal, yaitu *HazardEvent*, sebelum disimpan pada Canonical Store PostgreSQL dan didistribusikan sebagai event. Dengan demikian, service downstream tidak perlu memahami bentuk data khusus milik masing-masing instansi.

3.1.1 Aturan Pemetaan ke HazardEvent

Tabel 2 merangkum aturan pemetaan yang diterapkan oleh Aggregator. Field inti yang diperlukan oleh platform ditempatkan pada kolom kanonik, sedangkan detail sumber ditempatkan pada `attributes`.

Sumber	Data masukan	Pemetaan ke *HazardEvent*
BMKG	*SeismicEvent* dan *TsunamiWarning* terkait	`source=BMKG`, `source_ref_id=event_id`, `hazard_type=SEISMIC`. Nama wilayah dan koordinat diambil dari data gempa. Tingkat keparahan mengikuti peringatan tsunami bila tersedia; jika tidak tersedia, magnitude $\geq$ 6,5 dipetakan menjadi `SIAGA`, magnitude $\geq$ 5,0 menjadi `WASPADA`, dan sisanya `NORMAL`.
PVMBG	*VolcanicReport*	`source=PVMBG`, `source_ref_id=report_id`, `hazard_type=VOLCANIC`. `alert_level` dipetakan ke severity `NORMAL`, `WASPADA`, `SIAGA`, atau `AWAS`. Nama serta koordinat gunung diperoleh dari tabel referensi gunung api milik Aggregator.

Tabel 2: Aturan pemetaan data sumber ke format kanonik.

3.1.2 Kebijakan Evolusi Skema

Mekanisme yang dipilih adalah JSON dengan kebijakan *additive-only*. Field inti yang sudah dikenal dipetakan secara eksplisit untuk menentukan identitas hazard, jenis hazard, tingkat keparahan, lokasi, dan waktu kejadian. Setiap field tambahan dari sumber yang bukan field inti diteruskan tanpa transformasi ke objek `attributes`. Objek ini disimpan pada kolom `JSONB` di Canonical Store. Oleh karena itu, penambahan field seperti `confidence_level` oleh PVMBG tidak memerlukan perubahan kolom tabel, restart, maupun redeploy service.

Kebijakan penanganan field tak dikenal yang dideklarasikan pada sistem ini adalah **meneruskan field ke `attributes`**, bukan mengabaikannya. Konsistensi kebijakan tersebut dibuktikan melalui perubahan skema PVMBG; setelah field `confidence_level` diaktifkan, field tersebut tetap tersedia pada `attributes` dari *HazardEvent* hasil Aggregator.

3.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
Kontrak kanonik *HazardEvent*	Client dan consumer cukup memahami satu bentuk data meskipun sumber data heterogen.	Aggregator menjadi tempat pemeliharaan aturan mapping. Aturan dipisahkan pada mapper per sumber dan diuji secara terpisah.
JSONB dengan kebijakan *additive-only*	Field baru dapat diterima tanpa migrasi database ataupun redeploy; informasi sumber tidak hilang.	Validasi field dinamis lebih longgar dan query atribut tertentu lebih kompleks daripada kolom relasional. Field yang dibutuhkan oleh proses inti tetap dipertahankan sebagai field kanonik yang tervalidasi.
Mapping eksplisit untuk field inti, `attributes` untuk sisanya	Semantik inti seperti severity dan lokasi konsisten, sementara evolusi aditif tetap fleksibel.	Perubahan makna atau tipe field inti tidak dapat ditangani otomatis dan perlu perubahan kontrak serta mapper.

Tabel 3: Keputusan desain dan trade-off untuk interoperabilitas data.

Perubahan skema yang didukung secara sadar adalah penambahan field opsional pada objek laporan PVMBG dan BMKG. Perubahan yang tidak didukung tanpa perubahan kode adalah penghapusan, penggantian nama, perubahan tipe, atau perubahan makna dari field inti; misalnya `report_id`, `volcano_id`, `alert_level`, dan `reported_at` pada PVMBG. Sistem juga mengharapkan envelope respons tetap berbentuk `{data: [...]}`. Perubahan yang melanggar kontrak tersebut diperlakukan sebagai perubahan *breaking* dan harus disertai pembaruan mapper serta pengujian kompatibilitas.

3.3 Proof of Concept (PoC)

Langkah berikut digunakan untuk menunjukkan bahwa pemetaan kedua sumber berjalan dan perubahan skema aditif dapat diterima saat sistem aktif.

1. Nyalakan seluruh sistem dan pastikan setiap container sehat.

```
docker compose up -d --build
docker compose ps
```

2. Pastikan PVMBG menggunakan skema awal dengan menonaktifkan field tambahan. Endpoint admin ini hanya dipakai untuk demo/pengujian.

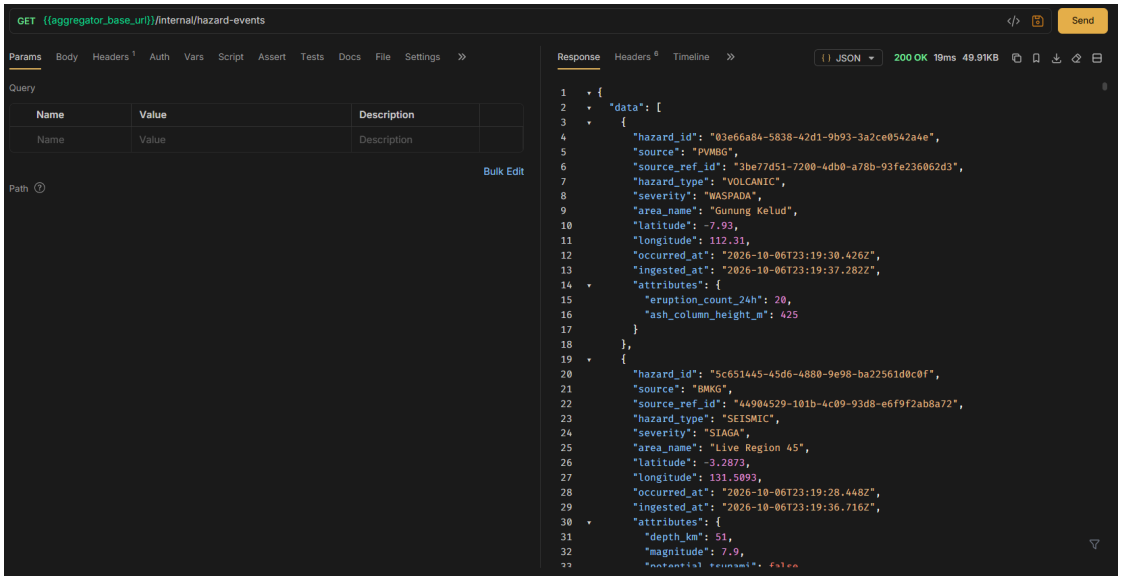
```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:8082/admin/schema-version `
  -ContentType "application/json" `
  -Body '{"enabled": false}'
```



Gambar 2: Status seluruh container setelah menonaktifkan field tambahan PVMBG.

3. Tunggu satu atau dua interval polling, kemudian ambil data kanonik dari API internal Aggregator. Respons harus memperlihatkan hazard dari kedua sumber dengan `source` masing-masing `BMKG` dan `PVMBG`.

```
$headers = @{ "X-Internal-Key" = "<INTERNAL_SERVICE_KEY dari .env>" }
Invoke-RestMethod `
  -Uri "http://localhost:8090/internal/hazard-events" `
  -Headers $headers
```



Gambar 3: Respons API internal Aggregator menampilkan data dari BMKG dan PVMBG.

4. Aktifkan perubahan skema saat semua service tetap berjalan. Tidak ada perintah restart atau redeploy setelah langkah ini.

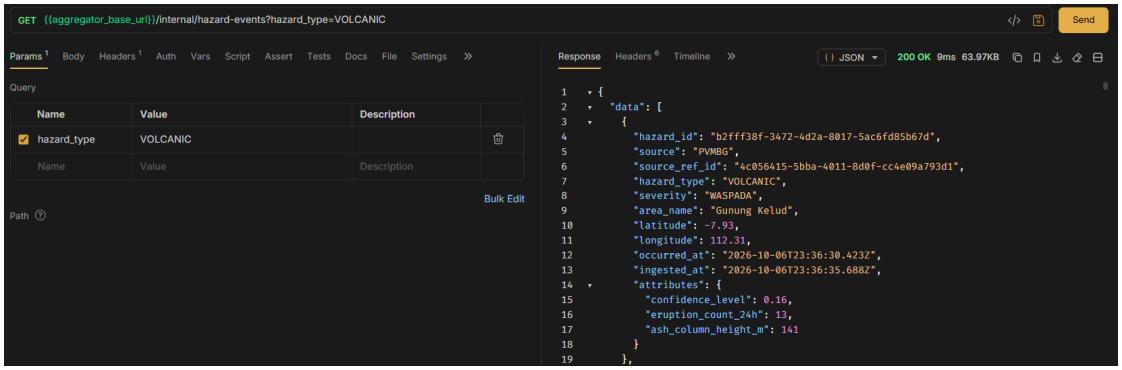
```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:8082/admin/schema-version `
  -ContentType "application/json" `
  -Body '{"enabled": true}'
```



Gambar 4: Mengaktifkan field tambahan PVMBG saat sistem berjalan.

5. Tunggu sedikitnya 15 detik agar PVMBG menghasilkan laporan baru dan Aggregator menyelesaikan polling. Ambil hanya data vulkanik, lalu verifikasi bahwa salah satu objek memiliki `attributes.confidence_level` tanpa ada perubahan schema tabel.

```
Invoke-RestMethod `
  -Uri "http://localhost:8090/internal/hazard-events?hazard_type=VOLCANIC" `
  -Headers $headers
```



Gambar 5: Respons API internal Aggregator setelah perubahan

6. Verifikasi bahwa Aggregator tetap sehat dan tidak mengalami crash.

```
docker compose ps aggregator
docker compose logs --tail=100 aggregator
```



Gambar 6: Perubahan scema version

Dengan proof tersebut, kriteria penyelesaian terpenuhi, yaitu sistem memetakan kedua sumber pada skema awal, menerima perubahan skema aditif ketika sedang berjalan, dan mempertahankan field tambahan secara konsisten di `attributes`.

4 Problem 2: Concurrency, Availability, dan Graceful Degradation

Pertanyaan Utama

Bagaimana sistem tetap responsif saat BMKG dan PVMBG memiliki latensi berbeda, menangani lonjakan permintaan, dan tetap melayani data yang tersedia ketika salah satu sumber mengalami outage?

4.1 Rancangan Solusi

Untuk mencegah kegagalan *head-of-line blocking*, *resource exhaustion*, dan *ketiadaan degradasi terkendali*, sistem kami menggunakan model konkurensi *asynchronous polling* yang dikombinasikan dengan penyimpanan lokal melalui *Canonical Store*, serta mekanisme *Circuit Breaker*.

Penjelasan pencegahan kegagalan:

1. **Head-of-line blocking**: Diatasi dengan memisahkan *polling loop* untuk setiap sumber (BMKG dan PVMBG). Setiap sumber berjalan pada *timer* yang independen (`PollingScheduler`),

sehingga keterlambatan dari PVMBG tidak akan memperlambat *polling* ke BMKG. Klien tidak memanggil API instansi eksternal secara berurutan dan sinkron; melainkan hanya membaca data dari *Canonical Store*.

2. **Resource exhaustion**: Diatasi dengan membatasi konkurensi per upstream menggunakan `ConcurrencyLimiter` serta pencegahan *polling* bertumpuk melalui validasi status `inFlight`. Beban puncak (*spike*) dari *client API* cukup dilayani oleh PostgreSQL sebagai *Canonical Store* yang mumpuni untuk operasi baca bersamaan, sehingga *upstream service* yang rentan tidak ikut kehabisan daya komputasi.
3. **Graceful Degradation**: Diimplementasikan lewat kombinasi `CircuitBreaker` dan pelayanan data *stale*. Jika PVMBG mati, *circuit breaker* akan memutus panggilan baru (*fail-fast*), merekam status sumber sebagai basi (`markSourceStale`), dan agregator tetap menyajikan rekaman terakhir yang ada di basis data. Respons yang mengandung data usang tersebut ditandai dengan *field* metadata eksplisit `stale_since`.

Konfigurasi dan Tooling Load Testing
Kami menggunakan **k6** untuk melakukan *load testing*. Pengujian dijalankan dengan total ≥ 50 *Virtual Users* (VUs) yang berjalan paralel dengan beban *sustained* selama 60 detik.

- Konfigurasi beban terbagi: 40 VUs khusus mengambil data BMKG (`SEISMIC`) dan 10 VUs memanggil data PVMBG (`VOLCANIC`).
- *Thresholds* diatur agar *error rate* tetap $< 1\%$ dan persentil ke-95 (p95) dari *latency* pemanggilan data BMKG-only terjamin berada di bawah 300 ms.

4.2 Alternatif dan Trade-off

- **Alternatif 1: Pemanggilan Paralel Sinkron (`Promise.all`) Tanpa Store**. *Trade-off*: Bebas dari *head-of-line blocking* komputasional, tetapi berisiko melipatgandakan beban pada instansi *upstream* setiap kali ada lonjakan permintaan, sehingga rentan *resource exhaustion*. Sangat lemah terhadap ketiadaan degradasi karena kegagalan instansi *upstream* tetap mematahkan operasi API klien.
- **Alternatif 2: Polling dengan In-Memory Cache**. *Trade-off*: Respon lebih cepat dan konfigurasi infrastruktur lebih sederhana. Kerugiannya, *cache* bersifat rentan terhadap kehilangan data historis apabila *aggregator* mengalami *restart* terencana maupun *crash* tiba-tiba. Penggunaan PostgreSQL sebagai Canonical Store mengorbankan sedikit *latency* baca namun unggul jauh pada persistensi.

4.3 Batasan dan Bukti Penyelesaian

Batasan yang disadari:
Sistem kami memiliki batasan pada tingkat kebaruan data (*freshness*). Model *polling* tidak bersifat *real-time event-driven*, melainkan tertunda oleh jeda interval *polling* (2-5 detik sesuai spesifikasi). Kami menukar kebaruan seketika dengan stabilitas tinggi pada arsitektur penyajian, mengisolasi *bottleneck* infrastruktur lambat PVMBG agar tidak mematikan sistem lain.

Bukti Penyelesaian:

Kami berhasil memvalidasi rancangan ini melalui eksekusi script `k6` (`load-tests/k6/hazard-events.js`). Meskipun PVMBG diatur memberikan *delay* konstan sebesar 3 detik (sesuai tuntutan *mock*), respons BMKG-only yang diukur oleh `k6` secara presisi terlaporkan berlatensi p95 kurang dari 300 ms. Saat *endpoint outage* dieksekusi, koneksi tidak lagi ditahan.

5 Problem 3: Service-to-Service Authentication dan Trust

Pertanyaan Utama

Bagaimana setiap domain kepercayaan dipisahkan sehingga kredensial antarsumber tidak dapat dipertukarkan, sementara hak akses client downstream tetap ditegakkan berdasarkan scope dan masa berlaku token?

5.1 Rancangan Solusi

Sistem memiliki tiga domain kepercayaan yang masing-masing memakai format kredensial dan secret yang berbeda:

1. **BMKG**: kunci statis pada header `X-BMKG-Key`, dicocokkan dengan `BMKG_API_KEY` di *bmkg-mock*. Bearer token ditolak dengan 403.
2. **PVMBG**: Bearer JWT HS256 dengan `aud: pvmbg`, ditandatangani `PVMBG_JWT_SECRET`. Token dibuat sekali dengan skrip `scripts/mint-pvmbg-token.js`. Kunci BMKG ditolak dengan 403.
3. **Client downstream**: Bearer JWT HS256 dengan `aud: bnpb-client`, ditandatangani `CLIENT_JWT_SECRET`, diterbitkan oleh *auth-service*. Claim berisi `sub`, `scope` (*media*, *lapangan*, atau *internal*), dan `purpose` (*access* atau *refresh*). TTL access token 60 detik dan refresh token 900 detik, keduanya dapat dikonfigurasi.

Tiga client downstream adalah identitas yang benar-benar berbeda, masing-masing dengan `client_id` dan `client_secret` sendiri dari environment. Perbedaan hak akses ditegakkan di sisi BNPB, bukan oleh flag di sisi client.

Auth-service bersifat stateless. Access dan refresh token adalah JWT yang memverifikasi dirinya sendiri, sehingga tidak ada tabel token. Refresh memeriksa bahwa client masih terdaftar dengan scope yang sama sebelum menerbitkan pasangan token baru.

Di *client-api*, middleware `auth.ts` memverifikasi JWT dan menyimpan scope pada request. Middleware `rbacFilter.ts` membatasi field: scope *media* hanya menerima field Ringkasan. Permintaan eksplisit `view=raw` dari scope *media* ditolak dengan 403 `FORBIDDEN_FIELD_ACCESS` sebelum panggilan ke Aggregator, sehingga data mentah tidak pernah diambil.

Setiap container hanya menerima variabel yang dibacanya. Tabel 4 merangkum pemetaannya. Sebelumnya seluruh container menerima `.env` penuh.

Service	Secret yang diterima
bmkg-mock	`BMKG_API_KEY`
pvmbg-mock	`PVMBG_JWT_SECRET`
aggregator	`BMKG_API_KEY`, `PVMBG_ACCESS_TOKEN`, `INTERNAL_SERVICE_KEY`
client-api	`CLIENT_JWT_SECRET`, `INTERNAL_SERVICE_KEY`
auth-service	`CLIENT_JWT_SECRET`, `AUTH_*_CLIENT_SECRET`
consumers, rabbitmq	`RABBITMQ_URL` atau `RABBITMQ_USER/PASSWORD`

Tabel 4: Pemetaan secret ke service. Client-api tidak menerima kunci BMKG, PVMBG, maupun kredensial auth.

5.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
JWT stateless (dipilih) dibanding token store (Redis atau Postgres)	Tidak ada dependensi tambahan, sesuai kontrak §6.2, dan tidak ada titik kegagalan baru.	Token tidak bisa dicabut sebelum `exp`. Dimitigasi dengan TTL pendek (60 detik).
Satu `CLIENT_JWT_SECRET` untuk semua scope (dipilih) dibanding secret per scope	Konfigurasi sederhana. Ketiga client berada di domain kepercayaan yang sama.	Kebocoran secret memungkinkan pemalsuan token scope mana pun. Dimitigasi dengan membatasi secret hanya ke auth-service dan client-api.
`environment` minimal per service (dipilih) dibanding `env_file` bersama	Kompromi satu container tidak membocorkan secret domain lain.	Daftar variabel per service harus dijaga manual saat ada perubahan.
Kunci statis untuk BMKG (dipilih) dibanding JWT	Sesuai kontrak. Sumber bersifat pasif dan ringan.	Kunci tidak punya masa berlaku dan harus dirotasi manual.
RBAC di client-api (dipilih) dibanding di Aggregator	Aggregator tetap menyimpan data lengkap untuk jalur internal. Filter satu tempat.	Client-api menjadi titik yang harus benar. Dimitigasi dengan unit test untuk filter dan scope.

Tabel 5: Keputusan desain dan trade-off untuk autentikasi dan trust.

5.3 Batasan yang Disadari

1. **Refresh token tidak dicabut setelah dipakai.** Refresh token lama tetap valid sampai `exp` (15 menit). Ini konsekuensi dari auth yang stateless (kontrak §6.2). Jika refresh token bocor, penyerang bisa memperbarui akses hingga 15 menit. Access token lama tetap ditolak setelah TTL 60 detik, sesuai spek.
2. **Penolakan token bergantung pada TTL, bukan blocklist.** Access token yang sudah dicabut secara manual tidak bisa dicabut sebelum kedaluwarsa.
3. **Satu secret untuk semua scope client.** Kompromi `CLIENT_JWT_SECRET` berarti token

scope *lapangan* atau *internal* bisa dipalsukan. Pemisahan per scope di luar cakupan M1.
4. **Secret tetap berupa environment variable.** Tidak ada secret manager, dan tidak ada rotasi otomatis. Client secret bersifat statis.
5. **Cakupan M1 hanya baca.** Ketiga client bersifat read-only, sehingga kompromi client belum bisa mengubah data di BNPB.

5.4 Bukti Penyelesaian

Bukti berikut diambil dari pengujian langsung terhadap sistem yang berjalan penuh lewat `docker compose up`, dijalankan 2026-10-07.

No	Skenario	Harapan	Hasil
1	Token *media*, `view=raw`	403	**403** (dikonfirmasi langsung). Kode `FORBIDDEN_FIELD_ACCESS` diverifikasi lewat unit test *client-api* (`app.test.ts`).
2	Token *media*, `GET /hazard-events`	Hanya field Ringkasan	**Sesuai** (dikonfirmasi langsung). Field yang muncul persis 7 field Ringkasan: `area_name`, `hazard_id`, `hazard_type`, `ingested_at`, `occurred_at`, `severity`, `source`. Tidak ada `source_ref_id`, `latitude`, `longitude`, atau `attributes`.
3	Token *lapangan*, tunggu lebih dari 60 detik, lalu gunakan	401	**401** (dikonfirmasi langsung), setelah menunggu 65 detik penuh terhadap TTL default 60 detik.
4	Refresh (`/auth/refresh`), lalu pakai token baru dan token lama	Token baru 200, token lama 401	Token lama: **401** (dikonfirmasi langsung). `POST /auth/refresh` menerbitkan `access_token` baru tanpa resubmit kredensial (dikonfirmasi langsung). Pemakaian token baru ke `/hazard-events` belum di-*re-test* eksplisit pada sesi ini; alur ini tercakup unit test *auth-service*.
5	Kunci BMKG ke PVMBG dan Bearer PVMBG ke BMKG	401 atau 403	`X-BMKG-Key` ke PVMBG: **403** (dikonfirmasi langsung). Kredensial berformat Bearer ke BMKG: **403** (dikonfirmasi langsung memakai token client; BMKG menolak setiap token berformat Bearer, termasuk token PVMBG asli – kasus dengan token PVMBG asli tercatat di *activity log* `2026-10-01_2020`).
6	Cek variabel di container *client-api* (`docker compose exec`)	Tidak memuat secret domain lain	**Sesuai** (dikonfirmasi langsung). Hanya nama variabel `CLIENT_JWT_SECRET` dan `INTERNAL_SERVICE_KEY` yang muncul. `BMKG_API_KEY`, `PVMBG_JWT_SECRET`, `PVMBG_ACCESS_TOKEN`, `AUTH_*_CLIENT_SECRET`, dan `DATABASE_URL` tidak ada.

Tabel 6: Skenario bukti Problem 3, diuji 2026-10-07.

Verifikasi otomatis: 18 unit test *auth-service* dan 28 unit test *client-api* lulus, termasuk filter RBAC, validasi scope, dan validasi parameter query.

5.5 Jawaban atas Butir "Di Laporan"

Mekanisme yang dipakai adalah JWT HS256 stateless dengan tiga trust domain terpisah. Mekanisme ini tidak membuka celah baru karena:

1. setiap domain memakai secret yang berbeda, sehingga token atau kunci satu domain tidak diterima domain lain (dibuktikan dengan skenario 5);
2. `aud` berbeda antara token client (`bnpb-client`) dan token PVMBG (`pvmbg`);
3. `purpose` membedakan access dan refresh, sehingga refresh token tidak bisa dipakai sebagai access token, dan sebaliknya;
4. secret hanya ada di container yang memerlukannya (tabel 4);
5. tidak ada kredensial yang di-hardcode atau di-commit. `.env` di-gitignore dan `.env.example` berisi placeholder.

Celah yang masih ada tercantum pada bagian Batasan.

6 Problem 4: Arsitektur Microservice dan Flexible Storage

Pertanyaan Utama

Bagaimana sistem dipecah menjadi layanan yang dapat dijalankan independen dan menyimpan atribut data yang berubah tanpa migrasi schema atau akses storage lintas service?

6.1 Rancangan Solusi

Untuk mencapai independensi sistem, aplikasi monolitik awal dipecah menjadi beberapa *microservice* (antara lain `aggregator`, `client-api`, dan `auth-service`), di mana setiap layanan dibungkus dalam *container* Docker terpisah via `docker-compose.yml`. Pembagian *boundaries* ditentukan berdasarkan fungsionalitas dan domain: `aggregator` dikhususkan pada beban *ingestion* dari *mock source* serta pengelolaan persistensi, sedangkan `client-api` berfungsi sebagai *API Gateway* untuk mengekspos endpoint ke luar dan memvalidasi akses klien.

Pada sisi penyimpanan (*flexible storage*), kami menggunakan **PostgreSQL**. Alih-alih mendefinisikan kolom statis untuk seluruh detail data, *Canonical Store* menempatkan atribut tambahan dan dinamis pada kolom `attributes` yang bertipe data **JSONB**. Dengan demikian, begitu PVMBG menerbitkan *field* baru seperti `confidence_level`, *field* tersebut langsung tersimpan ke dalam objek JSONB tanpa perlu melakukan proses *schema migration* pada basis data.

Selain itu, kepemilikan basis data sangat dibatasi. Basis data PostgreSQL hanya dapat diakses secara langsung oleh instansi `aggregator`. *Microservice* lain yang membutuhkan data kanonik (seperti `client-api`) wajib memanggil *internal REST API* milik `aggregator`.

6.2 Alternatif dan Trade-off

Kami membandingkan beberapa alternatif arsitektur penyimpanan:

- **Alternatif 1: Document Store murni (contoh: MongoDB)**. *Trade-off*: Menawarkan *schema-less* murni dan sangat tepat untuk dokumen berskema sangat dinamis, namun kami kehilangan dukungan kueri relasional bawaan. Hal ini dapat menghambat fleksibilitas pengembangan kedepan bila ada banyak join relasi struktural antar data internal yang ketat (misal data sumber vs data notifikasi).
- **Alternatif 2: Basis Data Relasional dengan Skema Konvensional (Kolom Baku)**. *Trade-off*: Paling ketat dalam menjaga validasi tipe data (*type-safe*). Namun, akan sangat menyulitkan operasi berjalan (menyebabkan interupsi sistem saat *deployment* dan migrasi tabel DDL) jika sumber eksternal mengubah *payload* laporan (mengandung kasus Problem 1).
- **Keputusan Akhir (PostgreSQL dengan kolom parsial JSONB)**: Kombinasi terbaik dari dua sisi. *Field* krusial untuk pencarian cepat seperti `hazard_type` dan `occurred_at` dipetakan sebagai kolom relasional yang diindeks, sementara data fleksibel diletakkan pada kolom JSONB `attributes`. Skema *record* campuran lama dan baru dapat berdampingan tanpa migrasi.

6.3 Batasan yang Disadari

Pemisahan *service* ini memperkenalkan tambahan latensi via lapisan jaringan (*HTTP call*) antara `client-api` dan `aggregator`, berbanding monolit konvensional. Penempatan data pencarian ke dalam blok JSONB juga mengurangi kinerja filter data jika kueri dijalankan pada komponen properti yang tertanam lebih dalam bila indeks *GIN* tidak dioptimalkan. Kami menerima kekurangan latensi mikro (*micro-latency*) demi mendapatkan ketahanan sistem terhadap perubahan dan kegagalan (*fault isolation*).

6.4 Bukti Penyelesaian

Kriteria terbukti ketika komponen berdiri independen dalam *container*, dan format JSONB menjaga kompatibilitas sebelum dan sesudah mutasi instansi sumber.

1. Verifikasi bahwa kode menginstruksikan *canonical-store* membuat kolom `attributes` berbasis tipe `JSONB`:

```
CREATE TABLE IF NOT EXISTS hazard_events (
    hazard_id UUID PRIMARY KEY,
    -- (kolom inti konvensional),
    attributes JSONB NOT NULL DEFAULT '{}'::jsonb
);
```

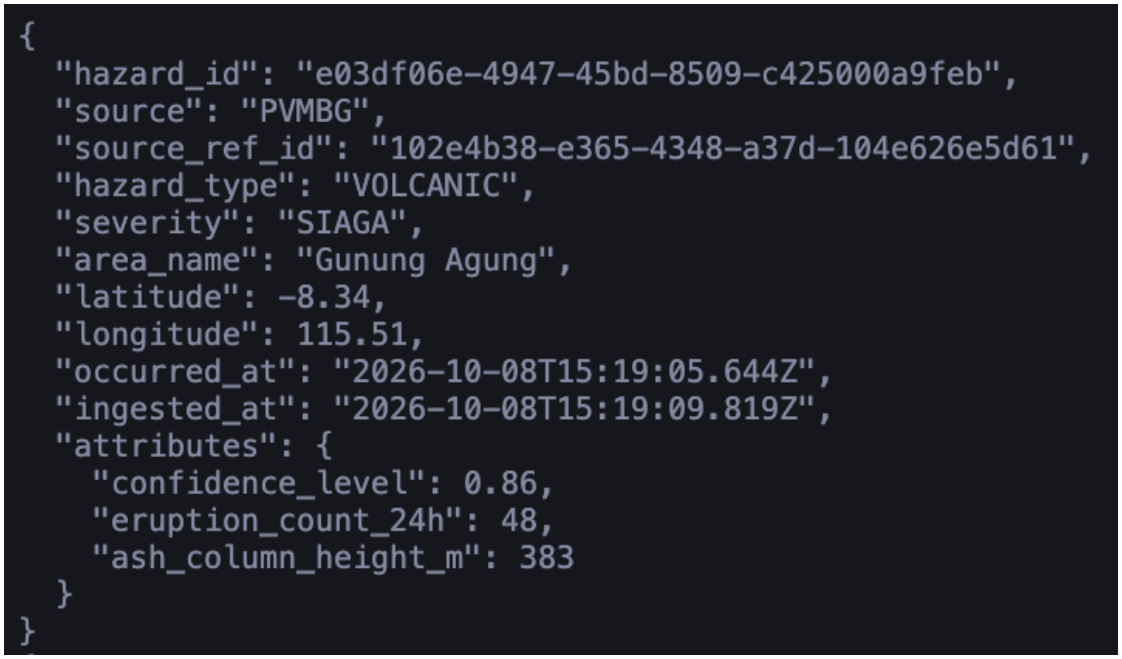
2. Verifikasi independensi *container* (bukti bahwa salah satu komponen dapat *restart* secara independen, tanpa menginterupsi *service* yang lain):

```
docker compose restart client-api
docker compose ps
```



Gambar 7: Bukti restart parsial sebuah container service

3. Verifikasi *co-existence* data lama dan baru (tanpa migrasi) berdampingan dalam JSONB pada *response payload*.



Gambar 8: Merekam log basis data (data lama berdampingan dengan confidence_level baru)

7 Problem 5: Fan-out Event Distribution dan Pub-sub

Pertanyaan Utama

Bagaimana HazardEvent baru didistribusikan ke banyak consumer tanpa producer memanggil consumer secara langsung atau terhambat ketika salah satu consumer mati?

7.1 Rancangan Solusi

Distribusi event memakai RabbitMQ 3.13 dengan satu exchange bertipe *fanout* bernama `hazard_events_exchange`. Setiap consumer memiliki queue durable sendiri yang terikat ke exchange tersebut: `dashboard_updater_queue` dan `notifier_queue`. Setiap queue menerima salinan penuh dari setiap event, sehingga consumer tidak bersaing memperebutkan pesan.

Aggregator mempublikasikan setiap HazardEvent baru melalui *publisher* yang menggunakan confirm channel. Pesan dikirim dengan flag persisten dan dibungkus dalam envelope `HazardEventMessage` yang memuat `message_id`, `correlation_id`, dan `payload`. `publish()`

hanya selesai setelah broker mengonfirmasi, dan gagal cepat (5 detik) jika broker tidak merespons. Publisher mendeklarasikan kedua queue saat start, sehingga event yang terbit sebelum consumer pertama kali menyala tetap tersimpan di queue.

Setiap consumer menjalankan langkah yang sama. Ack manual dengan prefetch 1 membuat pesan diproses satu per satu. Sebelum side effect dijalankan, consumer memeriksa `payload.hazard_id` di set dedup. Jika sudah pernah diproses, pesan langsung di-ack. Jika side effect gagal, pesan di-requeue. Envelope yang rusak dibuang.

7.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
RabbitMQ fanout (dipilih) dibanding Kafka	Model queue per consumer sederhana. Producer tidak tahu siapa consumer-nya. Cocok untuk skala M1.	Tidak ada log yang bisa diputar ulang oleh consumer baru. Event yang sudah di-ack tidak bisa diambil lagi.
Fanout (dipilih) dibanding topic atau consumer group	Setiap consumer menerima salinan penuh tanpa perlu routing key.	Consumer yang ingin subset harus memfilter sendiri.
At-least-once (dipilih) dibanding at-most-once (auto-ack)	Event tidak hilang saat consumer crash sebelum memproses. Lebih aman untuk peringatan evakuasi.	Duplikat mungkin terjadi. Ditangani dengan dedup dan idempotensi.
Dedup in-memory dengan batas 10.000 id (dipilih) dibanding dedup persisten	Tanpa dependensi tambahan. Cukup untuk M1.	Set hilang saat restart. Dijelaskan pada Batasan.
Pull dari broker (dipilih) dibanding HTTP push ke consumer	Consumer mengatur laju sendiri lewat prefetch. Producer tidak perlu tahu endpoint consumer.	Perlu broker tambahan yang harus selalu tersedia.

Tabel 7: Keputusan desain dan trade-off untuk distribusi event.

7.3 Batasan yang Disadari

1. **Tidak ada outbox.** Scheduler hanya mempublikasikan event yang baru disimpan (`created = true`). Jika broker tidak tersedia lebih dari 5 detik saat event itu terbit, event tersebut tidak dikirim ulang. Event tetap tersimpan di Canonical Store, tetapi consumer tidak menerimanya. Jalur perbaikan: kolom `published_at` pada `hazard_events` dan pengiriman ulang untuk baris yang belum terkirim.
2. **Dedup hilang saat restart.** Consumer yang crash tepat setelah memproses tetapi sebelum ack bisa menerima pesan yang sama lagi setelah restart. Pesan itu akan diproses dua kali. Semantik at-least-once tetap terjaga.
3. **Tidak ada dead-letter queue.** Pesan yang selalu gagal di side effect akan di-requeue berulang. Consumer tetap berjalan, tetapi pesan itu tidak pernah selesai.

4. **Kapasitas dedup terbatas.** Set menyimpan 10.000 hazard_id terakhir. Event yang lebih lama bisa diproses ulang jika muncul kembali sebagai duplikat.

7.4 Bukti Penyelesaian

Bukti berikut diambil dari pengujian langsung terhadap sistem yang berjalan penuh lewat `docker compose up`, dijalankan 2026-10-07, lengkap dengan `hazard_id` dan `correlation_id` asli dari log tiap service.

No	Skenario	Harapan	Hasil
1	Jalankan *dashboard-updater* dan *notifier*, publish satu event	Kedua consumer memproses event yang sama	**Sesuai.** `hazard_id 8c93bb08...` dan `correlation_id f0998e6a...` diproses kedua consumer pada detik yang sama (`09:06:24.511` dan `09:06:24.512`).
2	Hentikan *notifier*, publish 2 sampai 3 event	*dashboard-updater* tetap menerima, publisher tidak error	**Sesuai.** Selama 25 detik *notifier* mati, *dashboard-updater* tetap memproses 4 event baru (`hazard_id 4a73467b`, `929a2a56`, `0d2d37c0`, dan satu lagi) tanpa jeda atau error pada publisher.
3	Jalankan kembali *notifier*	Event yang tertunda tersusul dari queue	**Sesuai.** Begitu *notifier* tersambung kembali (`"message":"consuming"` pukul `09:06:54.303`), 6 event diproses beruntun dalam kurang dari 1 detik, termasuk 3 `hazard_id` yang sama persis dengan yang sudah diproses *dashboard-updater* saat *notifier* mati – bukti queue durable menahan pesan, bukan membuangnya. Lihat potongan log di bawah.
4	Jalankan *demo-logger* di topic yang sama	Menerima event baru tanpa perubahan producer	**Sesuai.** *demo-logger* langsung mencatat `bound, waiting for events`, lalu menerima 3 event baru (`hazard_id c26f8bbc`, `32f2a936`, `dc4737f5`) lengkap dengan `correlation_id` masing-masing, tanpa satu baris pun diubah di `event-publisher.ts`.

Tabel 8: Skenario bukti Problem 5, diuji 2026-10-07.

```
# Potongan log notifier tepat setelah tersambung kembali (skenario 3):
{"service":"notifier","queue":"notifier_queue","message":"consuming"}
{"service":"notifier","correlation_id":"3468dd8a-...","hazard_id":"4a73467b-...",
 "message":"notification dispatched for hazard event"}
{"service":"notifier","correlation_id":"56b6f062-...","hazard_id":"929a2a56-...",
 "message":"notification dispatched for hazard event"}
{"service":"notifier","correlation_id":"04196d74-...","hazard_id":"0d2d37c0-...",
 "message":"notification dispatched for hazard event"}
# Ketiga hazard_id di atas sama persis dengan yang sudah diproses dashboard-updater
# saat notifier mati, dan tersusul kurang dari 1 detik setelah notifier menyala.
```

Verifikasi otomatis: 13 unit test untuk setiap consumer lulus, termasuk dedup dan requeue.

7.5 Jawaban atas Butir "Di Laporan"

1. **Broker dan pola.** RabbitMQ dengan exchange fanout dan queue durable per consumer.
2. **Semantik pengiriman.** At-least-once. Pesan dipersistenkan oleh publisher dan di-ack manual oleh consumer setelah side effect berhasil.
3. **Konsekuensi bagi consumer.** Redelivery adalah kondisi normal, sehingga consumer wajib idempoten. Setiap consumer memeriksa `hazard_id` sebelum side effect. Kelompok menyikapinya dengan dedup in-memory untuk M1 dan mencatat batasannya.
4. **Consumer mati.** Producer tidak tertahan dan consumer lain tetap menerima. Event yang terbit selama consumer mati tersimpan di queue durable dan tersusul setelah consumer dijalankan kembali, selama broker tetap hidup.
5. **Consumer baru.** Consumer baru cukup mendeklarasikan queue sendiri dan mengikat ke exchange. Kode publisher tidak berubah.

A Bukti Pengujian Tambahan